

# An Adaptive Sharding-based Blockchain for Network Slicing in 5G

Quang Huy Do<sup>\*§</sup>, Sami Souihi<sup>†</sup>, Van Tong<sup>\*</sup>, Hai Anh Tran<sup>\*</sup>, Sara Tucci-Piergiovanni<sup>‡</sup>

<sup>\*</sup>School of Information and Communication Technology, Hanoi University of Science and Technology, Vietnam

<sup>†</sup>LISSI-TincNET Research Team, University of Paris-Est Creteil, France

<sup>‡</sup>Universite Paris-Saclay, CEA, List, F-91120, Palaiseau, France

<sup>§</sup> Texpect

Email: huy.dq184271@sis.hust.edu.vn, sami.souihi@u-pec.fr, vantv@soict.hust.edu.vn,  
anhth@soict.hust.edu.vn, sara.tucci@cea.fr

**Abstract**—Fifth-generation wireless technology, or 5G, promises increased data speeds, lower latency and greater capacity for mobile communications, enabling the growth of the Internet of Things (IoT). Network slicing is an advantageous feature of 5G that enables the creation of multiple virtual networks to meet the performance, security, and reliability needs of different services. This feature, combined with blockchain technology, provides secure and transparent data sharing between devices and networks. In addition, blockchain-enabled network slicing improves 5G network management and creates new business models for industries such as healthcare, transportation, and manufacturing. However, most current blockchain systems are limited in their ability to handle high throughput, which is not equivalent to what 5G can do. Therefore, sharding-based blockchain is proposed as a possible solution to improve the scalability and throughput of blockchain networks. By dividing the network into parts or shards, transactions can be processed simultaneously, allowing for faster transaction verification times and increased network capacity. Although current sharding-based blockchain networks have some limitations, such as static sharding policies that cannot cope with the dynamic blockchain environment, an adaptive sharding blockchain system using Deep Reinforcement Learning (DRL) methods has been proposed to address this situation. The approach allows the system to change or adapt the shard specifications, such as shard size or block size, whenever necessary to ensure maximum throughput while maintaining the security and integrity of the blockchain network.

**Index Terms**—5G, Blockchain, Network Slicing, Adaptive Sharding, Reinforcement Learning, security, privacy, trust.

## I. INTRODUCTION

The 5G mobile network [1] offers faster speeds, lower latency and higher capacity than before, enabling various applications and services that require reliability and high throughput. However, sharing resources among different parties is a challenge. Network slicing is an optimal solution that creates virtual networks for specific groups of customers or types of services. Each slice has its own policies and resources, enabling resource sharing while maintaining isolation and security. However, network slicing introduces new security, privacy and trust concerns, such as resource allocation and management, resource hoarding and abuse. Failure to properly allocate and manage resources can lead to network congestion, degraded performance, and reduced user satisfaction.

Blockchain can provide a decentralized, tamper-proof ledger of network resources. This can help ensure the efficient

allocation and management of resources, minimize network congestion, and prevent resource hoarding and misuse. For example, we can use blockchain as a distributed ledger to track the allocation and usage of resources in real time, which can help network operators identify available resources, allocate resources to different users and services based on their needs, and monitor the resource usage of each party in the network. As a result, if a malicious party monopolizes or misuses resources, the network administrator can easily identify it and take appropriate action. Despite these advantages, one of the major drawbacks of blockchain is its limited throughput. For example, bitcoin can process seven transactions per second, while Tezos processes about 350 transactions per second. These throughputs are still a far cry from what could be generated by a large-scale network like 5G. As a result, the application of blockchain to large-scale networks such as 5G faces many problems.

To improve blockchain performance, sharding has been used, which divides the system into smaller parts called shards. This allows multiple blocks to be created and validated in parallel, improving transaction throughput. However, most current sharding-based blockchains use fixed sharding policies, which can limit throughput even with high-capacity nodes. In addition, each shard may have different requirements, such as improving security or reducing resource consumption. To address these issues, we propose a blockchain framework that uses adaptive sharding to adjust sharding policies based on the blockchain system environment.

In this paper, we present an adaptive sharding-based blockchain using deep reinforcement learning (DRL). Our proposed protocol automatically adjusts shards based on the current state of the blockchain system, allowing for improved performance without compromising security. The main contributions of this paper are described below:

- The first contribution is to propose using blockchain in network slicing environment, which enhances transparency and trust in resource allocation and management.
- Then we introduce the adaptive sharding-based blockchain which adjusts sharding policy itself using Deep Reinforcement Learning (DRL) to meet each shard's requirements; hence, our proposed blockchain platform can adapt well to dynamic environment like network slicing in 5G while

optimizing performance.

The rest of this paper is organized as follows. Sections II and III provide background work on 5G, network slicing, resource sharing problems, blockchain, and sharding. Section IV provides an overview of our proposed system. Section V explains in detail how we implement our DRL-based adaptive sharding framework. The performance analysis for adaptive sharding-based blockchain networks is performed in Sec. VI. Finally, Section VI concludes the paper and highlights our future work.

## II. BACKGROUND

### A. 5G and Network Slicing

5G is the fifth generation of wireless communication tech that will change how we connect and communicate. It offers faster data transfer, lower latency, and increased capacity than previous generations. Used in applications like smart cities and industrial automation [2], 5G is transforming our interactions with the environment. One of its critical features is network slicing [3]. This allows multiple virtual networks to exist on a single physical network layer, and network operators can divide their resources into different logical networks. Each network slice has unique capabilities to meet the specific requirements of various users and applications, like low-latency support for autonomous vehicles or high-bandwidth support for video streaming. Network slicing enables the full potential of 5G networks by providing flexible and efficient resource allocation for various use cases.

### B. Resource sharing

Resource sharing is one of critical issue in 5G. It happens when the number of connected devices increases while available resources are limited. Resource sharing becomes a critical problem, especially in some smart systems with battery-constrained devices competing for resources from remote clouds. This results in network congestion, high delays, and poor performance. Therefore, there is a pressing need to develop effective resource allocation mechanisms that can distribute resources efficiently and ensure optimal network performance.

### C. Sharding

Sharding, according to related work, [4] [5], is one of the most promising solutions for the scalability problem. Originally, sharding [6] [7], is a type of database partitioning technique that separates a huge database into much smaller, faster, more easily managed parts called data shards. The key idea of sharding in the blockchain is to split the network into smaller committees. Each can independently process a disjoint set of transactions (or a “shard”). With sharding, nodes only have to communicate with nodes within its shard and only store the transaction happening in its shard.

### D. Tezos blockchain and its consensus algorithm

In this paper, we discuss the use of Tezos [8], a public and permissionless blockchain that is popular due to its self-amending mechanism. Tezos allows for updates without a hard

fork, making it a good fit for dynamic environments. The blockchain’s amendment mechanism, on-chain governance, is used to modify all bricks of Tezos, including the consensus algorithm. Recently, the consensus algorithm Emmy+ was replaced with Tenderbake, which prioritizes safety over liveness. Emmy+ [9] prioritizes liveness, which can be useful for fast transaction processing, but may cause issues with network instability. Tenderbake [10] pauses block production during asynchronous periods to ensure safety and security, making it better suited for situations where network stability is a concern.

## III. ADAPTIVE SHARDING-BASED BLOCKCHAIN FOR NETWORK SLICING IN 5G

### A. System overview

In this section, we present the system model and then describe the DRL-based sharding model in our system.

1) *System Model*: The system architecture, shown in Fig. 1, has several network slices for different uses, such as connected vehicles, personal computers, and surveillance cameras. Each slice has an independent blockchain network called a “shard”, which maintains several blockchain nodes. These blockchain nodes are a subset of network nodes of the corresponding slice. While network nodes refer to any device or entity that participates in our network slicing environment, blockchain nodes specifically pertain to devices that maintain a copy of the distributed ledger and participate in the consensus mechanism of the blockchain network. It’s possible for network nodes to be blockchain nodes, but not all network nodes necessarily function as blockchain nodes. Each shard in a slice has a corresponding leader node. All leaders from sub-shards maintain a main shard. The main shard only gives away sensitive data, like network topology, through a mutually accepted protocol, i.e. a smart contract, to protect data integrity. Each shard works independently and is tailored for a specific requirement of a slice, such as high safety for surveillance cameras or high bandwidth and throughput for vehicles. Therefore, the blockchain must have high throughput and a flexible policy to meet various application demands.

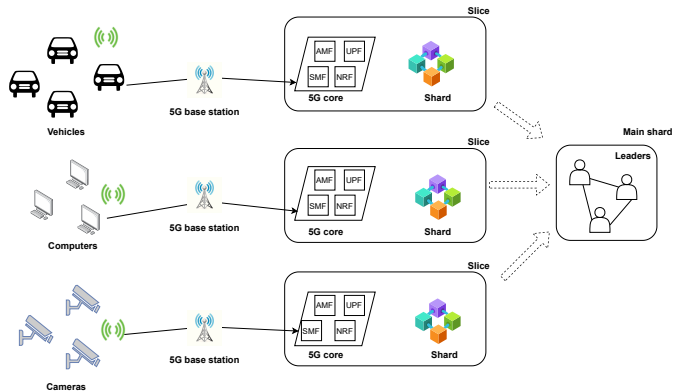


Fig. 1: Scalable blockchain for 5G network slicing management

2) *DRL-based Sharding Model*: Our approach takes into account the dynamic nature of the 5G applications environment. We design an adaptive sharding policy that balances performance and security based on the current system state using Deep Reinforcement Learning (DRL) approach.

Fig. 2 shows our framework. Epochs are the units of operation for our blockchain sharding system. The consensus and reconfiguration periods are the time intervals that comprise each epoch. The consensus period is when nodes propose and decide on a new block, while the reconfiguration period is when nodes in a shard decide on a sharding policy. At the end of epoch  $t$ , the protocol parameters (block size, nodes configuration including CPU and storage, consensus algorithm) and the number of nodes in each shard for the next epoch are used as the state  $s_t$  in DRL. Then the leaders of all shards at current epoch  $t$  use their agents to agree on an action  $a_t$ , i.e., shard size, block size, and consensus algorithm for the next epoch  $t + 1$ . After that, agents obtain a reward from the environment and store the transition  $(s_t; a_t; r_t; s_{t+1})$  into their buffers. At the same time, agents batch a constant number of previous transitions to update their neural networks, guaranteeing that agents can keep selecting an action to maximize the long-term reward regardless of the infinite dynamic environments.

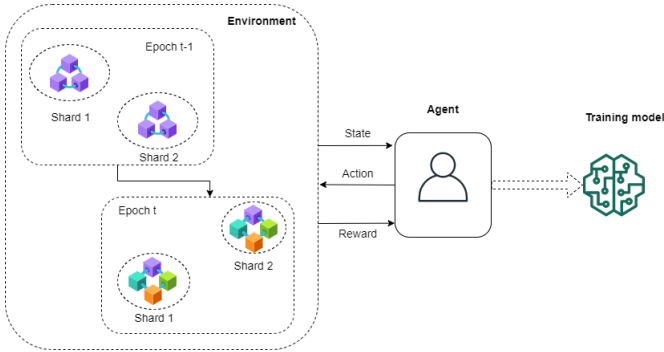


Fig. 2: DRL-based adaptive sharding framework.

## B. Problem statement

In this section, we propose a problem statement for the performance and security of the sharding systems. We measure the system's performance by calculating throughput, checking the relationship between node capacity and block size, and adjusting the consensus algorithm based on the shard requirement. Then, we analyze the security by considering a significant security problem in sharding systems, i.e., protocol health and shard size. Finally, considering efficiency and security, we define a sharding optimization problem.

### 1) Performance:

a) *Throughput*: We use Tezos blockchain, where the throughput can be calculated as follows:

$$tps = \frac{block\_size}{average\_transaction\_cost * minimal\_block\_delay} \quad (1)$$

where

- *block\_size* defines the maximum gas a block can carry or how many transactions that a block can store
- *average\_transaction\_cost* shows the average gas amount a transaction costs
- *minimal\_block\_delay* or the minimum interval between processing two consecutive blocks

b) *Node capacity*: The capacity of each node joining the shards is pre-defined, including CPU and storage. The larger the capacity of a node, the more blocks it can process and store. Therefore, if a shard has high-capacity nodes, the block size should be increased to utilize those powerful nodes. Conversely, if a shard contains low-capacity nodes, we should decrease the block size so the majority of nodes can participate. Then, the block size  $B_s$  and average node capacity  $A_{C_s}$  should have positive correlation relationship:

$$B_s \propto A_{C_s} \quad (2)$$

where

- $B_s$  is the block size of the shard.
- $A_{C_s}$  is the average capacity of a node in a shard.

c) *Shard requirement*: In 5G application, each network slice may have unique requirements at different times. These requirements include maximizing throughput, ensuring safety, or preventing network instability. For example, during periods of high network congestion, some slices may prioritize throughput to ensure transactions are processed efficiently. At other times, safety may be prioritized to prevent errors or malicious activity. Then, changing the consensus algorithm of a shard is reasonable, and  $C_s$  can be defined as:

$$C_{shard} = \begin{cases} Tenderbake & \text{if require safety} \\ Emmy+ & \text{if require liveness} \end{cases} \quad (3)$$

### 2) Security:

a) *Protocol health*: In a blockchain network, each node has a balance called  $Ba$  used to pay for storage and transaction fees. Malicious behavior like double-spending may result in a pre-defined punishment  $P$ . Nodes that contribute to the consensus process may receive rewards  $R$ .

A fixed punishment or balance amount may not be enough to maintain shard stability. When the average node balance is high but the reward and punishment amounts are low, nodes may not be incentivized to participate honestly. To address this, adjust the reward and punishment mechanisms based on the average balance of nodes in a shard. Define reward and punishment constraints based on Tezos protocol parameters [11].

$$0 < P_s \leq 10\%Ba_s \quad (4)$$

$$0 < R_s \leq 0.05\%Ba_s \quad (5)$$

b) *Shard size*: The Byzantine fault tolerance in most PBFT-based consensus is 1/3. Therefore we consider the shard will become unsafe if there are more than 1/3 malicious nodes  $mf_s$  over  $m_s$  total nodes. We think it is secure enough if the following inequality is satisfied:

$$mf_s < 1/3 * m_s \quad (6)$$

3) *Problem Formulation*: The performance of the blockchain sharding system can be measured by transaction throughput. Then, the problem formulation is as follows:

$$\begin{aligned} \max \quad & tps \\ \text{s.t.} \quad & mf_s < 1/3 * m_s \\ & B_s \propto A_{C_s} \\ & 0 < P_s \leq 10\% Ba_s \\ & 0 < R_s \leq 0.05\% Ba_s \\ & C_{shard} = \begin{cases} Tenderbake & \text{if require safety} \\ Emmy+ & \text{if require liveness} \end{cases} \end{aligned} \quad (7)$$

#### IV. DRL-BASED ADAPTIVE SHARDING FRAMEWORK

Our proposed solution to address the trade-off issue in dynamic blockchain systems is to utilize a deep reinforcement learning approach, which can make our platform adaptive to complex and dynamic blockchain environments. Since our action space is continuous, we employ Deep Deterministic Policy Gradient (DDPG) algorithm, a Deep Reinforcement Learning technique, to train our model.

##### 1) DRL Model Design:

a) *State space*: The protocol parameters, including number of nodes, block size, punishment, reward, the average balance of each node, and the requirement of each shard, are denoted as  $q$ . The average transaction cost is denoted as  $A_{C_s}$ . Thus, the state space at epoch  $t$  can be expressed by:

$$s_t = [q, A_{C_s}]_t \quad (8)$$

b) *Action space*: As discussed above, shard size  $m$ , block size  $B$ , reward  $R$ , punishment  $P$ , and consensus algorithm  $C$  play a part in tackling the problem of trade-offs in the sharding system and changing the blockchain environment. The block size should be adjusted based on the shard size and capacity of nodes in the shard. Thus, they should be adjusted to adapt to the dynamic environment. We define the action space at epoch  $t$  as:

$$a_t = [m, B, R, P, C]_t \quad (9)$$

c) *Reward space*: Our optimization target is to obtain a sharding policy that could balance performance and security in every epoch. Since the scalability can be quantified easily with transaction throughput, we use transaction throughput as our reward function. And we have several constraints regarding the relationship between block size and average capacity, the relationship between punishment, reward, and average balance of each node, and finally, each shard requirement.

Then, the reward  $rt$  in epoch  $t$  can be defined as:

$$r_t(a_t, s_t) = \begin{cases} tps & \text{if (2) - (6) are satisfied} \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

---

#### Algorithm 1 DRL-based Adaptive Sharding Framework

---

- 1: **Initialize** Critic network  $Q(s; a|\theta^Q)$  and Actor network  $\mu(s|\theta^\mu)$  with weights  $\theta^Q$  and  $\theta^\mu$ .
- 2: **Initialize** target critic network and target actor network with weights  $\theta^{Q'} \leftarrow \theta^Q$  and  $\theta^{\mu'} \leftarrow \theta^\mu$ .
- 3: **Initialize** Target network update rate  $\tau$ , learning rate  $\eta$ , and replay buffer  $R$ .
- 4: **Set** training episode  $E$  and training steps  $T$ .
- 5: **for**  $episode = 1$  to  $E$  **do**
- 6:   Initialize an exploration noise  $N$  for action exploration;
- 7:   Initialize environment and receive initial observation state  $s_1$ ;
- 8:   **for**  $epoch t = 1$  to  $T$  **do**
- 9:     Select action  $a_t = \mu(s_t, \theta^\mu)$  according to  $s_t$  and the current policy;
- 10:    Execute  $a_t$  to set next reward, punishment, shard size, consensus algorithm and block size;
- 11:    **if** all constraints are satisfied **then**
- 12:     Set reward  $r_t$  to be throughput;
- 13:    **else**
- 14:     Set reward  $r_t = 0$ ;
- 15:    **end if**
- 16:    Observe next state  $s_{t+1}$  and check if *done*;
- 17:    Store transition  $(s_t, a_t, r_t, s_{t+1})$  in  $R$ ;
- 18:    Sample a random minibatch of  $N$  transitions  $(s_i, a_i, r_i, s_{i+1})$  from  $R$ ;
- 19:    Set  $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}), \theta^{Q'})$ ;
- 20:    Update critic by minimizing the loss:  $L_C = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$ ;
- 21:    Update the actor policy using the policy gradient:  $\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i (\nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i})$ ;
- 22:    Update the target networks:  $\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta_{Q'}$ ,  $\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta_{\mu'}$ ;
- 23:    **end for**
- 24: **end for**

---

2) *DRL Training Methodology*: Algorithm 1 outlines the training methodology of our DRL-based adaptive sharding framework. The agent uses two networks, an actor network  $\mu(s|\theta^\mu)$  and a critic network  $Q(s, a, \theta^Q)$  (Line 1). Target networks initialized from the actor and critic networks are used to calculate target values for network updates (Line 2). A replay buffer ensures independent and identically distributed training samples (Line 3).

At each timestep, the agent selects an action  $s_t$  based on the current state and executes it to receive a reward  $s_t$  from the environment, transitioning to the next state  $s_{t+1}$  (Line 12-15). The transition  $(s_t, a_t, r_t, s_{t+1})$  is stored in the replay buffer. A batch of transitions is used to calculate the loss function  $L_C$

| Parameter                                                   | Value |
|-------------------------------------------------------------|-------|
| The number of blockchain shards, $s_{shard}$                | 3     |
| The initial number of blockchain nodes per shard, $p$       | 4     |
| The maximum number of blockchain nodes per shard, $p_{max}$ | 100   |
| The number of network slices, $s_{slice}$                   | 3     |
| The minimal block delay, $D$                                | 15s   |
| The initial number of network nodes, $q$                    | 10    |
| The maximum number of network nodes, $q_{max}$              | 100   |
| The Byzantine fault tolerance, $f$                          | 1/3   |
| The maximum block size, $B_{max}$                           | 11MB  |
| The maximum reward, $R_{max}$                               | 20    |
| The maximum punishment, $P_{max}$                           | 2000  |

TABLE I: Simulation Parameters

and policy gradient  $\nabla_{\theta\mu} J$  for updating the network parameters. Finally, DDPG employs "soft" target updates to slowly modify the target networks, ensuring a stable and efficient learning process.

3) *Training process*: At the beginning of every epoch, current leaders (denoted as  $L_s$ ) of each shard are randomly selected as a sharding policy proposer. Nodes that want to become leaders have to stake a pre-defined amount of tokens (6000 tez). And if the leader behaves maliciously, we adopt a voting mechanism [12] so that we can replace them. After a cycle that lasts 2048 blocks,  $L_s$  uses the information of the current system state (i.e., the system parameters  $q$  and average transaction cost of last cycle  $A_C$  as the input of its agent to make a sharding policy. After the learning process, each leader broadcasts the sharding policy to other nodes of its shard.

## V. PERFORMANCE EVALUATION

In this section, we conduct an analysis on the convergence of performance, meeting of constraints and compare the metrics of our proposed platform with two other platforms. We used Tezos-k8s to deploy Tezos blockchain infrastructure on a Kubernetes cluster for testing. Besides, we use Tensorflow to build our neural networks and implemented our algorithm with Python 3.7 in Ubuntu 20.04. The parameters setting used in the simulations is summarized in Table I.

### A. Convergence analysis

We use the reward function in Eq. 10 to assess the transactions throughput. As shown in Fig. 3, our proposed DRL-based adaptive sharding scheme has a convergence performance superior to that of the Tezos blockchain. Tezos blockchain, which runs on Emmy plus consensus algorithm with fixed parameters (2MB block size, 4 nodes, i.e., ...), has fixed throughput (346 TPS) as calculated by Eq.1. In comparison, we

can observe that the throughput of our scheme rapidly increases from the beginning of the learning process and becomes stable at approximately 1900 TPS after around 200 training episodes. At this point, our platform's number of nodes increases from 12 to 75, where each shard has respectively from 4 to 25 blockchain nodes, and the block size increase to 5MB with the Tenderbake consensus algorithm. This demonstrates that the DRL-based adaptive sharding algorithm can determine the appropriate sharding policy based on different network conditions to achieve a long-term reward in our sharding system. The results in Fig. 3 also indicate the superior performance of our proposed scheme compared to that of the Tezos blockchain.

### B. Constraints analysis

After each episode, the shard size, block size, reward, punishment and consensus algorithm are adjusted to maximize throughput while also keep system secured.

a) *Reward and punishment*: Figures 4 and 5 show the change in reward and punishment as the average balance of each node in a shard gradually increases. The orange area indicates the valid range of reward and punishment. It shows that after the learning process, the amount of reward and punishment satisfies our requirement (4) (5).

b) *Block size with node capacity*: the context of blockchain, nodes with different CPU and memory capacities can freely join and leave. To optimize performance, the block size should be adjusted to the average capacity of the nodes. As shown in figures 6 and 7, the learning process confirms the expected positive correlation between block size and CPU/memory.

### C. Performance Comparison of Key Metrics Across Platforms

Finally, in this section, we assess the interest of a blockchain-based approach for managing slicing, after tackling the barriers of consensus protocols inherent in blockchain that could limit its adoption in 5G use cases. We compare approaches for managing resource sharing of 5G slices with and without blockchain. The blockchain-based solution is based on our adaptive sharing mechanism, which, as shown in the previous results, is suitable for the needs of this application. Our example consists of a 5G network with three slices. It is important to note that the term "network node" in this context does not refer to a blockchain node. The first non-blockchain-based approach we want to compare is the fair resource sharing model, where each slice receives the same amount of resources. The last platform we want to compare is the best-effort platform, which provides network resources on a best-effort basis without any specific guarantees or priorities. This approach, by definition, does not limit a particular slice's use of resources. However, it can be limited to ensure greater fairness. Nevertheless, without a relevant governance mechanism to ensure there is no cheating, network nodes have the potential to behave maliciously by consuming more bandwidth than necessary, leading to network congestion, degraded performance, and reduced user satisfaction. This is why a blockchain-based solution could be a suitable solution.

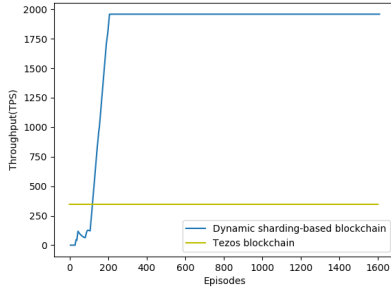


Fig. 3: Convergence performance of two schemas

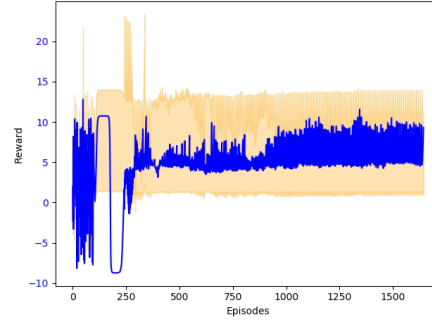


Fig. 4: Reward with episodes.

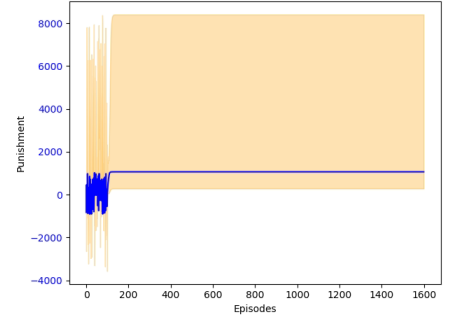


Fig. 5: Punishment with episodes.

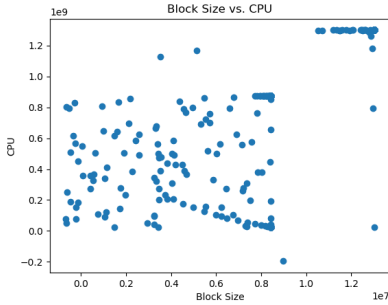


Fig. 6: Block size with CPU.

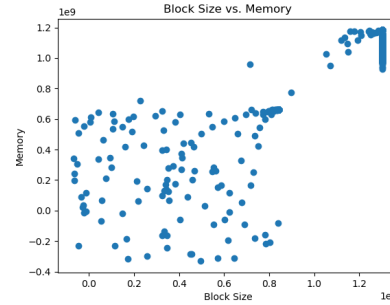


Fig. 7: Block size with Memory.

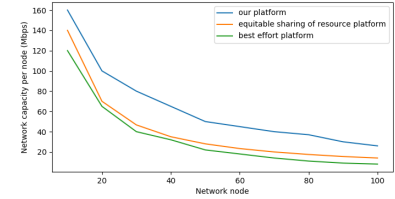


Fig. 8: Network capacity per node with number of network nodes.

Fig. 8 shows the effect of node count on network capacity (Mbps) across three platforms. Our adaptive sharding-based platform, which adjusts sharding policies according to network environments, performs the best. With 100 nodes, approximately 10% participate in the blockchain, maximizing resource allocation and management. In contrast, platforms without blockchain face challenges such as congestion, degraded performance, and reduced user satisfaction. Our platform dynamically adjusts slice bandwidth to achieve the best performance while keeping the network functioning normally. The network capacity per node reaches around 26 Mbps, while equitable resource sharing and best-effort platforms only reach 14 and 8 Mbps, respectively.

## VI. CONCLUSION

This paper focuses on network slicing and its resource sharing issue in 5G networks. This challenge can be partially addressed by using blockchain technology, but it is necessary to develop a specific platform that can dynamically adjust itself. Therefore, we propose an adaptive sharding-based blockchain. Our system uses a DRL-based approach to adjust block size, protocol health, and shard requirement, maintaining a balance between performance and security. This allows it to be well adapted to network slices. In conclusion, the integration of blockchain technology with 5G networks can have a significant impact on the security of the network, especially by mitigating the risk of malicious use. Moreover, blockchain opens a new research area in network operator business models, which we will focus on in future research.

## REFERENCES

- [1] A. Gupta and R. K. Jha, "A survey of 5g network: Architecture and emerging technologies," *IEEE access*, vol. 3, pp. 1206–1232, 2015.
- [2] B. Hamdaoui, M. Alkalbani, A. Rayes, and N. Zorba, "Iotshare: A blockchain-enabled iot resource sharing on-demand protocol for smart city situation-awareness applications," *IEEE Internet of Things Journal*, vol. 7, no. 10, pp. 10 548–10 561, 2020.
- [3] D. C. Nguyen, P. N. Pathirana, M. Ding, and A. Seneviratne, "Blockchain for 5g and beyond networks: A state of the art survey," *Journal of Network and Computer Applications*, vol. 166, p. 102693, 2020.
- [4] A. Hafid, A. S. Hafid, and M. Samih, "Scaling blockchains: A comprehensive survey," *IEEE Access*, vol. 8, pp. 125 244–125 262, 2020.
- [5] H. Nam Nguyen, H. Anh Tran, S. Fowler, and S. Souihi, "A survey of blockchain technologies applied to software-defined networking: Research challenges and solutions," *IET Wireless Sensor Systems*, vol. 11, no. 6, pp. 233–247, 2021.
- [6] G. Wang, Z. J. Shi, M. Nixon, and S. Han, "Sok: Sharding on blockchain," in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, 2019, pp. 41–61.
- [7] Y. Liu, J. Liu, M. A. V. Salles, Z. Zhang, T. Li, B. Hu, F. Henglein, and R. Lu, "Building blocks of sharding blockchain systems: Concepts, approaches, and open problems," *Computer Science Review*, vol. 46, p. 100513, 2022.
- [8] V. Allombert, M. Bourgoïn, and J. Tesson, "Introduction to the tezos blockchain," in *2019 International Conference on High Performance Computing & Simulation (HPCS)*. IEEE, 2019, pp. 1–10.
- [9] Nomadic Labs. Emmy+: an improved consensus algorithm. Accessed May, 20, 2022. [Online]. Available: <https://research-development.nomadic-labs.com/emmy-an-improved-consensus-algorithm.html>
- [10] S. Conchon, A. Korneva, Ç. Bozma, M. Iguernlala, and A. Mebsout, "Formally documenting tenderbake," *Open Access Series in Informatics*, 2021.
- [11] Nomadic Labs, "Tezos Consensus Documentation," 2021, accessed: May 8, 2023. [Online]. Available: <https://tezos.gitlab.io/alpha/consensus.html#sharding>
- [12] M. Castro and B. Liskov, "Practical byzantine fault tolerance and proactive recovery," *ACM Transactions on Computer Systems (TOCS)*, vol. 20, no. 4, pp. 398–461, 2002.